

Spring Boot Security & OAuth2

Complete In-Depth Guide – From Scratch

Authentication • Authorisation • JWT • OAuth2 Flows

Social Login • Resource Server • Authorization Server

Method Security • CSRF • CORS • Best Practices

Clear Definitions • Architecture Diagrams • Full Code Examples

Table of Contents

1. Spring Security Fundamentals

- 1.1 What is Spring Security?
- 1.2 Core Concepts – Authentication vs Authorisation
- 1.3 Security Filter Chain
- 1.4 SecurityContext & SecurityContextHolder
- 1.5 Maven Setup

2. Form Login & HTTP Basic Authentication

- 2.1 Default Security Behaviour
- 2.2 Custom UserDetailsService
- 2.3 PasswordEncoder – BCrypt
- 2.4 Custom Login Form
- 2.5 Remember-Me Authentication
- 2.6 Session Management

3. Authorisation – Roles & Permissions

- 3.1 Role-Based Access Control (RBAC)
- 3.2 URL-Based Authorisation
- 3.3 Method-Level Security
- 3.4 @PreAuthorize & @PostAuthorize
- 3.5 @Secured & @RolesAllowed
- 3.6 Domain Object Security (ACL overview)

4. JWT – JSON Web Token Authentication

- 4.1 What is JWT?
- 4.2 JWT Structure – Header.Payload.Signature
- 4.3 Stateless Authentication Flow
- 4.4 Implementing JWT from Scratch
- 4.5 JWT Filter – OncePerRequestFilter
- 4.6 Refresh Tokens

5. OAuth2 Fundamentals

- 5.1 What is OAuth2?
- 5.2 OAuth2 Roles
- 5.3 OAuth2 Grant Types
- 5.4 Authorization Code Flow (with PKCE)
- 5.5 Client Credentials Flow
- 5.6 Tokens – Access, Refresh, ID

6. Spring Boot OAuth2 Client – Social Login

- 6.1 OAuth2 Login Overview
- 6.2 Google Login Setup
- 6.3 GitHub Login Setup
- 6.4 Custom OAuth2 User Service
- 6.5 Handling OAuth2 Success/Failure

7. Spring Boot Resource Server

- 7.1 Resource Server Concepts
- 7.2 JWT Resource Server Setup
- 7.3 Opaque Token Introspection
- 7.4 Custom JWT Claims & Authorities
- 7.5 Securing REST Endpoints

8. Spring Authorization Server

- 8.1 What is Spring Authorization Server?
- 8.2 Setting Up the Auth Server
- 8.3 Registered Clients
- 8.4 Issuing JWT Tokens
- 8.5 PKCE Flow End-to-End

9. CSRF, CORS & Security Best Practices

- 9.1 CSRF Protection
- 9.2 CORS Configuration
- 9.3 Security Headers
- 9.4 Common Vulnerabilities & Fixes

CHAPTER 1

Spring Security Fundamentals

Architecture, Security Filter Chain, Authentication & Authorisation concepts

1.1 What is Spring Security?

Spring Security

Spring Security is a powerful, highly customisable authentication and access-control framework for Java applications. It is the de-facto standard for securing Spring-based applications. It provides comprehensive support for authentication (who are you?), authorisation (what are you allowed to do?), protection against common exploits (CSRF, session fixation, clickjacking), and integration with OAuth2/OIDC.

- **Authentication** – Verifying the identity of a user (login with username/password, token, certificate).
- **Authorisation** – Determining what an authenticated user is allowed to access.
- **Servlet Filters** – Spring Security works as a chain of servlet filters that intercept every request.
- **Declarative Security** – Secure URLs and methods using annotations and configuration without touching business code.

1.2 Core Architecture

Security Filter Chain

FilterChainProxy

Spring Security registers a FilterChainProxy servlet filter with the Servlet container. This proxy delegates to a chain of Spring Security filters in a specific order. Each filter handles one concern – CSRF protection, session management, authentication, exception handling, etc.

Filter	Responsibility
SecurityContextPersistenceFilter	Loads/saves SecurityContext to/from HttpSession
UsernamePasswordAuthenticationFilter	Processes /login form POST; authenticates credentials
BasicAuthenticationFilter	Processes HTTP Basic Authorization header
BearerTokenAuthenticationFilter	Processes Bearer JWT/opaque tokens (OAuth2 Resource Server)
ExceptionTranslationFilter	Catches AuthenticationException / AccessDeniedException
FilterSecurityInterceptor	Enforces access control on URLs
CsrfFilter	Validates CSRF tokens on state-changing requests
CorsFilter	Handles CORS pre-flight and response headers
LogoutFilter	Processes /logout requests

Key Interfaces & Classes:

Interface / Class	Role
SecurityContext	Holds the Authentication object for the current thread
SecurityContextHolder	Static holder; provides access to SecurityContext anywhere
Authentication	Represents the authenticated principal + credentials + authorities
UserDetails	Core user info – username, password, authorities, account status
UserDetailsService	Loads UserDetails by username (implement this for DB auth)
GrantedAuthority	Represents a permission/role granted to the principal
AuthenticationManager	Processes an Authentication object; delegates to providers
AuthenticationProvider	Performs specific authentication (DAO, LDAP, JWT)
PasswordEncoder	Encodes and matches passwords (BCryptPasswordEncoder)
SecurityFilterChain	The configured chain of filters for URL patterns

1.3 Maven Setup

pom.xml dependencies

```
<!-- pom.xml -->
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- For JWT support -->
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.12.3</version>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId>
<version>0.12.3</version>
<scope>runtime</scope>
</dependency>
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId>
<version>0.12.3</version>
<scope>runtime</scope>
</dependency>
```

```
<!-- For OAuth2 Client + Resource Server -->
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

CHAPTER 2

Form Login & HTTP Basic Authentication

UserDetailsService, BCrypt, Custom Login, Remember-Me & Session Management

2.1 Default Security & SecurityFilterChain Configuration

- When spring-boot-starter-security is on the classpath, Spring Boot auto-configures basic security: all URLs are protected, a default login page is generated, and a random password is printed to the console at startup.

SecurityFilterChain – Form Login

```
// Minimal custom SecurityFilterChain – Spring Security 6.x
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**", "/login", "/register").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login") // custom login page URL
                .loginProcessingUrl("/login") // POST endpoint that processes credentials
                .defaultSuccessUrl("/dashboard", true)
                .failureUrl("/login?error=true")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout=true")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
            );
        return http.build();
    }
}
```

2.2 Custom UserDetailsService

UserDetailsService

UserDetailsService is the interface you implement to load user data from any source (database, LDAP, in-memory). Spring Security calls loadUserByUsername() during authentication. You return a UserDetails object containing the username, encoded password, and list of GrantedAuthority objects.

Custom UserDetailsService with DB

```
// User Entity
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String password; // always store BCrypt-hashed
    private String role; // e.g. ROLE_USER, ROLE_ADMIN
    private boolean enabled = true;
    // getters, setters
}

// UserDetailsService implementation
@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepo;

    public CustomUserDetailsService(UserRepository userRepo) {
        this.userRepo = userRepo;
    }

    @Override
    public UserDetails loadUserByUsername(String username)
        throws UsernameNotFoundException {
        User user = userRepo.findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("User not found: " + username));

        return org.springframework.security.core.userdetails.User
            .withUsername(user.getUsername())
            .password(user.getPassword()) // must be BCrypt encoded
            .roles(user.getRole()) // adds ROLE_ prefix automatically
            .accountExpired(!user.isEnabled())
            .credentialsExpired(false)
            .disabled(!user.isEnabled())
            .build();
    }
}
```


2.3 PasswordEncoder – BCrypt

BCryptPasswordEncoder

Passwords must NEVER be stored in plain text. BCrypt is the recommended algorithm: it is a slow adaptive hashing function with a built-in salt, making brute-force attacks computationally expensive. Spring Security's PasswordEncoder interface has encode() and matches() methods.

BCryptPasswordEncoder

```
// Register the PasswordEncoder as a Bean
@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12); // strength 12 (default 10)
    }
}

// Encoding password at registration
@Service
public class RegistrationService {
    private final UserRepository repo;
    private final PasswordEncoder encoder;

    public RegistrationService(UserRepository repo, PasswordEncoder encoder) {
        this.repo = repo; this.encoder = encoder;
    }

    public void register(String username, String rawPassword, String role) {
        User user = new User();
        user.setUsername(username);
        user.setPassword(encoder.encode(rawPassword)); // hash before saving
        user.setRole(role);
        repo.save(user);
    }
}

// Manual password check
boolean matches = encoder.matches("rawPassword", storedHash);
```

2.4 Session Management

Session Management & Remember-Me

```
// Session management in SecurityFilterChain
.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED) // default
    // SessionCreationPolicy.STATELESS – for JWT REST APIs (no session)
    // SessionCreationPolicy.ALWAYS – always create session
    // SessionCreationPolicy.NEVER – never create, use if exists
    .maximumSessions(1) // only 1 concurrent session per user
```

```
.maxSessionsPreventsLogin(false) // kick old session (true = block new login)
.expiredUrl("/login?expired=true")
)

// Remember-Me authentication
.rememberMe(remember -> remember
.key("my-secret-remember-me-key")
.tokenValiditySeconds(7 * 24 * 60 * 60) // 7 days
.rememberMeParameter("remember-me") // checkbox name in form
)
```

CHAPTER 3

Authorisation – Roles & Permissions

URL-Based Rules, Method Security, @PreAuthorize, @PostAuthorize & SpEL

3.1 URL-Based Authorisation with requestMatchers

URL-Based Authorisation Rules

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(auth -> auth

        // Public endpoints – no login required
        .requestMatchers("/", "/home", "/about").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()

        // Role-based access
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .requestMatchers("/manager/**").hasAnyRole("ADMIN", "MANAGER")

        // Authority-based (use hasAuthority for exact string match)
        .requestMatchers("/reports/**").hasAuthority("GENERATE_REPORT")

        // Authenticated user (any role)
        .requestMatchers("/profile/**").authenticated()

        // All other requests need authentication
        .anyRequest().authenticated()
    );
    return http.build();
}
```

3.2 Method-Level Security

Method Security

Method-level security lets you place access-control annotations directly on service methods. Spring wraps the method in an AOP proxy and evaluates the security expression before (Pre) or after (Post) execution. Enable with `@EnableMethodSecurity`.

`@PreAuthorize`, `@PostAuthorize`, `@Secured` examples

```
// Enable method security on @Configuration class
@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true)
public class SecurityConfig { ... }

@Service
public class EmployeeService {
```

```

// Only users with ROLE_ADMIN can call this
@PreAuthorize("hasRole('ADMIN')")
public void deleteEmployee(Long id) {
    employeeRepo.deleteById(id);
}

// User can view their own record; ADMIN can view any
@PreAuthorize("hasRole('ADMIN') or #id == authentication.principal.id")
public Employee getEmployee(Long id) {
    return employeeRepo.findById(id).orElseThrow();
}

// Filter the returned list - only return own records unless ADMIN
@PostAuthorize("hasRole('ADMIN') or returnObject.createdBy == authentication.name")
public Report getReport(Long reportId) {
    return reportRepo.findById(reportId).orElseThrow();
}

// Filter collection returned by method
@PostFilter("hasRole('ADMIN') or filterObject.owner == authentication.name")
public List<Document> getAllDocuments() {
    return documentRepo.findAll();
}

// @Secured - simpler, no SpEL, just role names
@Secured({"ROLE_ADMIN", "ROLE_MANAGER"})
public List<Employee> getAll() {
    return employeeRepo.findAll();
}
}

```

3.3 SpEL Expressions in Security

SpEL Expression	Meaning
hasRole('ADMIN')	User has ROLE_ADMIN (ROLE_ prefix auto-added)
hasAnyRole('ADMIN','MANAGER')	User has any of the listed roles
hasAuthority('READ_DATA')	User has exact authority string READ_DATA
hasAnyAuthority('READ','WRITE')	User has any of listed authorities
isAuthenticated()	User is logged in (not anonymous)
isAnonymous()	User is NOT logged in
isFullyAuthenticated()	Logged in without remember-me token
principal.username == 'admin'	Authenticated principal's username
#paramName == authentication.name	Method param equals logged-in username
@myBean.check(authentication,#param)	Delegate check to a custom Spring bean

CHAPTER 4

JWT – JSON Web Token Authentication

Stateless Auth, Token Structure, Custom Filter & Refresh Tokens

4.1 What is JWT?

JSON Web Token (JWT)

A JWT is a compact, URL-safe, self-contained token used to securely transmit information between parties as a JSON object. The information is digitally signed (HMAC SHA256 or RSA) so it can be verified and trusted. JWTs enable stateless authentication: the server doesn't need to store session data – all user info is encoded in the token itself.

JWT Structure – three Base64URL-encoded parts separated by dots:

JWT Structure

```
xxxxx.yyyyy.zzzzz

# HEADER (algorithm + type)
{
  "alg": "HS256",
  "typ": "JWT"
}

# PAYLOAD (claims)
{
  "sub": "1234567890", -- Subject (user ID)
  "name": "Alice Smith",
  "email": "alice@example.com",
  "roles": ["ROLE_USER"],
  "iat": 1700000000, -- Issued At (Unix timestamp)
  "exp": 1700003600 -- Expiration (1 hour later)
}

# SIGNATURE
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secret_key
)
```

■ The payload is only Base64-encoded, NOT encrypted. Never store sensitive data (passwords, credit cards) in a JWT payload. Anyone with the token can decode and read it. The signature only guarantees integrity, not confidentiality.

4.2 Stateless Authentication Flow

- **Step 1 – Login:** Client POSTs username/password to /api/auth/login.
- **Step 2 – Verify:** Server validates credentials against the database.
- **Step 3 – Issue Token:** Server generates a signed JWT with user info and expiry.
- **Step 4 – Return Token:** Server returns {accessToken, refreshToken} as JSON.
- **Step 5 – Client Stores:** Client stores token (memory or httpOnly cookie).
- **Step 6 – Subsequent Requests:** Client sends Authorization: Bearer <token> header.
- **Step 7 – Filter Validates:** JwtAuthFilter extracts token, validates signature, sets SecurityContext.
- **Step 8 – Request Proceeds:** Controller executes; no session lookup needed.

4.3 Full JWT Implementation

JWT properties

```
// application.properties
jwt.secret=myVeryLongSuperSecretKeyThatIsAtLeast256BitsLong!@#$
jwt.expiration=3600000 # 1 hour in ms
jwt.refresh-expiration=604800000 # 7 days in ms
```

JwtUtils – Token Generation & Validation

```
// JwtUtils – token generation and validation
@Component
public class JwtUtils {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private long expirationMs;

    private SecretKey getSigningKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secret);
        return Keys.hmacShaKeyFor(keyBytes);
    }

    // Generate access token
    public String generateToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("roles", userDetails.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority).collect(Collectors.toList()));

        return Jwts.builder()
            .claims(claims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(getSigningKey())
            .compact();
    }
}
```

```

// Extract username from token
public String extractUsername(String token) {
    return extractClaim(token, Claims::getSubject);
}

// Check token validity
public boolean isTokenValid(String token, UserDetails userDetails) {
    final String username = extractUsername(token);
    return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
}

private boolean isTokenExpired(String token) {
    return extractExpiration(token).before(new Date());
}

private Date extractExpiration(String token) {
    return extractClaim(token, Claims::getExpiration);
}

public <T> T extractClaim(String token, Function<Claims, T> resolver) {
    Claims claims = Jwts.parser()
        .verifyWith(getSigningKey())
        .build()
        .parseSignedClaims(token)
        .getPayload();
    return resolver.apply(claims);
}
}

```

JwtAuthFilter – OncePerRequestFilter

```

// JWT Authentication Filter – runs on every request
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtUtils jwtUtils;
    private final UserDetailsService userDetailsService;

    public JwtAuthFilter(JwtUtils jwtUtils, UserDetailsService uds) {
        this.jwtUtils = jwtUtils; this.userDetailsService = uds;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");

        // Skip if no Bearer token
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            chain.doFilter(request, response);
            return;
        }
    }
}

```



```

final String token = authHeader.substring(7);
final String username = jwtUtils.extractUsername(token);

// Only authenticate if not already authenticated
if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {

    UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

    if (jwtUtils.isTokenValid(token, userDetails)) {
        UsernamePasswordAuthenticationToken authToken =
new UsernamePasswordAuthenticationToken(
userDetails, null, userDetails.getAuthorities());
        authToken.setDetails(
new WebAuthenticationDetailsSource().buildDetails(request));
        SecurityContextHolder.getContext().setAuthentication(authToken);
    }
}

chain.doFilter(request, response);
}
}

```

SecurityConfig – Stateless JWT

```

// SecurityConfig for JWT stateless REST API

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final UserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable()) // stateless - no CSRF needed
            .sessionManagement(s -> s
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .anyRequest().authenticated()
            )
            // Add JWT filter BEFORE the default username/password filter
            .addFilterBefore(jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean

```

```

public AuthenticationManager authenticationManager(
    AuthenticationConfiguration config) throws Exception {
    return config.getAuthenticationManager();
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}

```

AuthController – Login & Register

```

// Auth Controller - login & register
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authManager;
    private final CustomUserDetailsService uds;
    private final JwtUtils jwtUtils;
    private final RegistrationService regService;

    // POST /api/auth/login
    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest req) {
        // Authenticate - throws BadCredentialsException if invalid
        authManager.authenticate(
            new UsernamePasswordAuthenticationToken(req.username(), req.password()));

        UserDetails userDetails = uds.loadUserByUsername(req.username());
        String token = jwtUtils.generateToken(userDetails);
        return ResponseEntity.ok(new JwtResponse(token));
    }

    // POST /api/auth/register
    @PostMapping("/register")
    @ResponseStatus(HttpStatus.CREATED)
    public void register(@RequestBody @Valid RegisterRequest req) {
        regService.register(req.username(), req.password(), "USER");
    }
}

// DTOs
public record LoginRequest(String username, String password) {}
public record RegisterRequest(
    @NotBlank String username,
    @Size(min=8) String password) {}
public record JwtResponse(String accessToken) {}

```

CHAPTER 5

OAuth2 Fundamentals

What is OAuth2, Roles, Grant Types, PKCE, Tokens & OpenID Connect

5.1 What is OAuth2?

OAuth 2.0

OAuth 2.0 is an open authorisation framework (RFC 6749) that enables a third-party application to obtain limited access to an HTTP service on behalf of a resource owner (user), without exposing the user's credentials. Instead of sharing passwords, OAuth2 issues access tokens that grant specific scopes of access.

OpenID Connect (OIDC)

OpenID Connect is an identity layer built on top of OAuth2. While OAuth2 handles authorisation ('what can this app do?'), OIDC handles authentication ('who is the user?'). OIDC adds an ID Token (JWT) that contains the user's identity information, and a /userinfo endpoint.

5.2 OAuth2 Roles

Role	Definition	Example
Resource Owner	The user who owns the data and grants access	End-user (you)
Client	The application requesting access on behalf of the user	Your Spring Boot app
Authorization Server	Issues tokens after authenticating the resource owner	Google, GitHub, Keycloak
Resource Server	Hosts the protected resources; validates access tokens	Your REST API backend

5.3 OAuth2 Grant Types (Flows)

Grant Type	Use Case	Who Uses It
Authorization Code	Web apps where user logs in via browser	Web apps (most common)
Authorization Code + PKCE	Mobile/SPA apps; prevents code interception attacks	Mobile, SPA (recommended)
Client Credentials	Machine-to-machine; no user involved	Backend services, daemons
Device Code	Devices with no browser (Smart TV, CLI)	IoT, CLI apps
Refresh Token	Obtain new access token without re-login	Any flow that issues refresh tokens

Grant Type	Use Case	Who Uses It
Implicit (deprecated)	Was used for SPAs – insecure, use PKCE instead	Legacy – do not use
Password (deprecated)	First-party apps only – insecure, avoid	Legacy – do not use

5.4 Authorization Code Flow with PKCE – Step by Step

- **1. App generates PKCE:** code_verifier (random string) and code_challenge = SHA256(code_verifier).
- **2. Redirect to Auth Server:** App sends user to /authorize?response_type=code &client_id=...&redirect_uri=...&scope=openid email&code_challenge=...&code_challenge_method=S256.
- **3. User Authenticates:** Auth server shows login page; user enters credentials.
- **4. Consent:** Auth server shows consent screen ('Allow this app to...').
- **5. Auth Code Returned:** Auth server redirects back to redirect_uri?code=AUTH_CODE.
- **6. Token Exchange:** App POSTs to /token with code, client_id, redirect_uri, code_verifier.
- **7. Auth Server Verifies PKCE:** Recalculates challenge from verifier; matches stored value.
- **8. Tokens Issued:** Auth server returns {access_token, refresh_token, id_token, expires_in}.
- **9. API Calls:** App uses access_token as Bearer token in Authorization header.
- **10. Refresh:** When access_token expires, POST /token with grant_type=refresh_token.

5.5 Token Types

Token	Format	Purpose	Lifetime
Access Token	JWT or Opaque	Authorise API calls; sent in Authorization header	Short: 5 min – 1 hr
Refresh Token	Opaque string	Obtain new access token; stored securely by client	Long: days – weeks
ID Token	JWT (OIDC)	Identify the user; contains sub, name, email claims	Short: same as access
Authorization Code	Short opaque	One-time code exchanged for tokens; expires in seconds	Very short: 30–60 sec

CHAPTER 6

Spring Boot OAuth2 Client – Social Login

Google, GitHub OAuth2 Login, Custom OAuth2UserService & Success Handler

6.1 How OAuth2 Login Works in Spring Boot

spring-boot-starter-oauth2-client

When you add the `oauth2-client` starter, Spring Boot auto-configures OAuth2 login. Spring registers `/oauth2/authorization/{registrationId}` endpoints (e.g. `/oauth2/authorization/google`). Clicking 'Login with Google' redirects there, which starts the Authorization Code flow. After Google redirects back, Spring exchanges the code for tokens and loads user info.

Maven dependency

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

6.2 Google OAuth2 Login Setup

- Go to `console.cloud.google.com` → APIs & Services → Credentials.
- Create an OAuth 2.0 Client ID; Application type = Web application.
- Add Authorised redirect URI: `http://localhost:8080/login/oauth2/code/google`
- Copy the Client ID and Client Secret.

OAuth2 Client Registration in properties

```
# application.properties
spring.security.oauth2.client.registration.google.client-id=YOUR_GOOGLE_CLIENT_ID
spring.security.oauth2.client.registration.google.client-secret=YOUR_GOOGLE_SECRET
spring.security.oauth2.client.registration.google.scope=openid,email,profile

# GitHub
spring.security.oauth2.client.registration.github.client-id=YOUR_GITHUB_CLIENT_ID
spring.security.oauth2.client.registration.github.client-secret=YOUR_GITHUB_SECRET
spring.security.oauth2.client.registration.github.scope=read:user,user:email
```

SecurityConfig with OAuth2 Login

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
```

```

http
.authorizeHttpRequests(auth -> auth
.requestMatchers("/", "/login").permitAll()
.anyRequest().authenticated()
)
.oauth2Login(oauth2 -> oauth2
.loginPage("/login") // custom login page
.defaultSuccessUrl("/dashboard", true)
.failureUrl("/login?error=true")
.userInfoEndpoint(ui -> ui
.userService(customOAuth2UserService) // custom user service
)
);
return http.build();
}
}

```

Custom OAuth2UserService – Save User to DB

```

// Custom OAuth2UserService – save/update user in DB
@Service
public class CustomOAuth2UserService implements OAuth2UserService<OAuth2UserRequest,
OAuth2User> {

private final UserRepository userRepo;

@Override
public OAuth2User loadUser(OAuth2UserRequest userRequest)
throws OAuth2AuthenticationException {

// Load basic user info from provider
DefaultOAuth2UserService delegate = new DefaultOAuth2UserService();
OAuth2User oAuth2User = delegate.loadUser(userRequest);

String provider = userRequest.getClientRegistration().getRegistrationId();
String email = oAuth2User.getAttribute("email");
String name = oAuth2User.getAttribute("name");

// Upsert user in our database
User user = userRepo.findByEmail(email)
.orElseGet(() -> {
User u = new User();
u.setEmail(email);
u.setProvider(provider);
u.setRole("ROLE_USER");
return u;
});
user.setName(name);
userRepo.save(user);

return new DefaultOAuth2User(
Collections.singleton(new SimpleGrantedAuthority(user.getRole())),

```

```
oAuth2User.attributes(),  
"email");  
}  
}
```

CHAPTER 7

Spring Boot Resource Server

Protecting REST APIs with JWT Bearer Tokens & Token Introspection

7.1 Resource Server Concepts

Resource Server

A Resource Server is a server that hosts protected resources (REST API endpoints). It validates incoming Bearer tokens on each request and allows or denies access based on the token's claims and scopes. Spring Boot provides first-class support for both JWT tokens (validated locally using public key) and opaque tokens (validated by calling the Authorization Server's introspection endpoint).

7.2 JWT Resource Server Setup

Resource Server dependency

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
```

Resource Server JWT properties

```
# application.properties - point to Authorization Server JWKS URI
# Spring Boot fetches public keys and validates JWT signatures automatically
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://accounts.google.com

# OR specify JWKS URI directly
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=https://accounts.google.com/.well-known/
jwks.json

# OR use a local secret key (for self-issued JWT)
spring.security.oauth2.resourceserver.jwt.secret-key=mySecretKey256bits...
```

Resource Server SecurityConfig with JWT

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class ResourceServerConfig {

  @Bean
  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
      .csrf(csrf -> csrf.disable())
      .sessionManagement(s -> s
        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
  }
}
```



```

.authorizeHttpRequests(auth -> auth
.requestMatchers("/api/public/**").permitAll()
.requestMatchers("/api/admin/**").hasAuthority("SCOPE_admin")
.anyRequest().authenticated()
)
.oauth2ResourceServer(oauth2 -> oauth2
.jwt(jwt -> jwt
.jwtAuthenticationConverter(jwtAuthConverter())
)
);
return http.build();
}

// Convert JWT claims to Spring Security authorities
@Bean
public JwtAuthenticationConverter jwtAuthConverter() {
    JwtGrantedAuthoritiesConverter authConverter =
    new JwtGrantedAuthoritiesConverter();
    authConverter.setAuthoritiesClaimName("roles"); // custom claim name
    authConverter.setAuthorityPrefix("ROLE_");

    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(authConverter);
    return converter;
}
}

```

CHAPTER 8

Spring Authorization Server

Build Your Own OAuth2 / OIDC Authorization Server

8.1 What is Spring Authorization Server?

Spring Authorization Server

Spring Authorization Server (SAS) is an officially supported framework for building OAuth2-compliant Authorization Servers in Spring. It implements the full OAuth2 and OIDC specifications: Authorization Code (with PKCE), Client Credentials, Refresh Token, Token Introspection, Token Revocation, and OpenID Connect UserInfo endpoint.

Spring Authorization Server dependency

```
<!-- pom.xml -->
<dependency>
<groupId>org.springframework.security</groupId>
<artifactId>spring-security-oauth2-authorization-server</artifactId>
<version>1.2.0</version>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

8.2 Complete Authorization Server Configuration

Authorization Server Configuration

```
@Configuration
@EnableWebSecurity
public class AuthServerConfig {

    // 1. Authorization Server security filter chain

    @Bean
    @Order(1)
    public SecurityFilterChain authServerFilterChain(HttpSecurity http) throws Exception {
        OAuth2AuthorizationServerConfiguration.applyDefaultSecurity(http);
        http.getConfigurer(OAuth2AuthorizationServerConfigurer.class)
            .oidc(Customizer.withDefaults()); // enable OIDC
    }
}
```

```

http
.exceptionHandling(ex -> ex
.defaultAuthenticationEntryPointFor(
new LoginUrlAuthenticationEntryPoint("/login"),
new MediaTypeRequestMatcher(MediaType.TEXT_HTML)))
.oauth2ResourceServer(rs -> rs.jwt(Customizer.withDefaults()));
return http.build();
}

// 2. Default security chain for login page etc.
@Bean
@Order(2)
public SecurityFilterChain defaultFilterChain(HttpSecurity http) throws Exception {
http
.authorizeHttpRequests(auth -> auth.anyRequest().authenticated())
.formLogin(Customizer.withDefaults());
return http.build();
}

// 3. RSA key pair for signing JWTs
@Bean
public JWKSource<SecurityContext> jwkSource() {
KeyPair keyPair = generateRsaKey();
RSAPublicKey publicKey = (RSAPublicKey) keyPair.getPublic();
RSAPrivateKey privateKey = (RSAPrivateKey) keyPair.getPrivate();
RSAKey rsaKey = new RSAKey.Builder(publicKey)
.privateKey(privateKey)
.keyID(UUID.randomUUID().toString())
.build();
JWKSet jwkSet = new JWKSet(rsaKey);
return new ImmutableJWKSet<>(jwkSet);
}

private static KeyPair generateRsaKey() {
try {
KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");
gen.initialize(2048);
return gen.generateKeyPair();
} catch (Exception ex) { throw new RuntimeException(ex); }
}

// 4. JWT decoder and encoder
@Bean
public JwtDecoder jwtDecoder(JWKSource<SecurityContext> jwkSource) {
return OAuth2AuthorizationServerConfiguration.jwtDecoder(jwkSource);
}

@Bean
public JwtEncoder jwtEncoder(JWKSource<SecurityContext> jwkSource) {

```

```

return new NimbusJwtEncoder(jwkSource);
}

// 5. Authorization server settings (issuer URI)
@Bean
public AuthorizationServerSettings authorizationServerSettings() {
return AuthorizationServerSettings.builder()
.issuer("http://localhost:9000")
.build();
}
}

```

Registered Clients – Web App & Service

```

// 6. Registered OAuth2 clients
@Configuration
public class ClientConfig {

@Bean
public RegisteredClientRepository registeredClientRepository() {

// Client 1: Web app using Authorization Code + PKCE
RegisteredClient webClient = RegisteredClient
.withId(UUID.randomUUID().toString())
.clientId("web-client")
.clientSecret("{noop}web-secret") // {noop} = no encoding
.clientAuthenticationMethod(ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
.authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
.authorizationGrantType(AuthorizationGrantType.REFRESH_TOKEN)
.redirectUri("http://localhost:8080/login/oauth2/code/my-auth-server")
.scope(OidcScopes.OPENID)
.scope(OidcScopes.PROFILE)
.scope("read")
.scope("write")
.clientSettings(ClientSettings.builder()
.requireProofKey(true) // require PKCE
.requireAuthorizationConsent(true)
.build())
.build();

// Client 2: Backend service using Client Credentials
RegisteredClient serviceClient = RegisteredClient
.withId(UUID.randomUUID().toString())
.clientId("service-client")
.clientSecret("{noop}service-secret")
.clientAuthenticationMethod(ClientAuthenticationMethod.CLIENT_SECRET_BASIC)
.authorizationGrantType(AuthorizationGrantType.CLIENT_CREDENTIALS)
.scope("api.read")
.scope("api.write")
.build();
}
}

```

```
return new InMemoryRegisteredClientRepository(webClient, serviceClient);  
}  
}
```

Auth Server User Store & Properties

```
// 7. User store for authentication  
@Configuration  
public class UserConfig {  
  
    @Bean  
    public UserDetailsService userDetailsService() {  
        UserDetails admin = User.withDefaultPasswordEncoder()  
            .username("admin")  
            .password("password")  
            .roles("ADMIN")  
            .build();  
        UserDetails user = User.withDefaultPasswordEncoder()  
            .username("user")  
            .password("password")  
            .roles("USER")  
            .build();  
        return new InMemoryUserDetailsManager(admin, user);  
    }  
  
    @Bean  
    public PasswordEncoder passwordEncoder() {  
        return new BCryptPasswordEncoder();  
    }  
}  
  
# application.properties for Auth Server  
server.port=9000
```

CHAPTER 9

CSRF, CORS & Security Best Practices

Protection Against Common Vulnerabilities, Security Headers & Production Hardening

9.1 CSRF Protection

CSRF (Cross-Site Request Forgery)

CSRF is an attack where a malicious website tricks a logged-in user's browser into making an unwanted request to your server. Spring Security enables CSRF protection by default for browser-based apps. For stateless REST APIs using JWT (no cookies/session), CSRF can be safely disabled.

CSRF Configuration

```
// CSRF enabled (default) - for form-based web apps
http.csrf(csrf -> csrf
    .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
    // Sends XSRF-TOKEN cookie; JS reads it and sends back as X-XSRF-TOKEN header
);

// CSRF disabled - for stateless JWT REST APIs
http.csrf(csrf -> csrf.disable());

// Ignore CSRF for specific paths (e.g. webhooks)
http.csrf(csrf -> csrf
    .ignoringRequestMatchers("/webhook/**")
);
```

9.2 CORS Configuration

CORS (Cross-Origin Resource Sharing)

CORS is a browser security policy that prevents JavaScript on one origin from making requests to a different origin. When your React frontend (localhost:3000) calls your Spring Boot API (localhost:8080), the browser sends a CORS pre-flight OPTIONS request. Your server must respond with the appropriate Access-Control-Allow-* headers.

CORS Configuration

```
// Method 1: Global CORS in SecurityConfig (recommended)
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.cors(cors -> cors.configurationSource(corsConfigurationSource()));
    // ... other config
    return http.build();
}
@Bean
```

```

public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of(
        "http://localhost:3000",
        "https://myfrontend.com"
    ));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Requested-With"));
    config.setExposedHeaders(List.of("Authorization"));
    config.setAllowCredentials(true);
    config.setMaxAge(3600L); // pre-flight cache for 1 hour

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}

// Method 2: @CrossOrigin on individual controllers
@RestController
@CrossOrigin(origins = "http://localhost:3000", maxAge = 3600)
public class ApiController { ... }

```

9.3 Security Headers

Security Headers Configuration

```

// Spring Security adds these headers by default:
// X-Content-Type-Options: nosniff
// X-Frame-Options: DENY
// X-XSS-Protection: 0
// Cache-Control: no-cache, no-store, max-age=0
// Strict-Transport-Security: max-age=31536000 (HTTPS only)

// Customising headers
http.headers(headers -> headers
    .frameOptions(frame -> frame.sameOrigin()) // allow iframes from same origin
    .contentSecurityPolicy(csp -> csp
        .policyDirectives(
            "default-src 'self'; " +
            "script-src 'self' https://cdn.jsdelivr.net; " +
            "style-src 'self' https://fonts.googleapis.com; " +
            "img-src 'self' data: https;")
        )
    )
    .httpStrictTransportSecurity(hsts -> hsts
        .maxAgeInSeconds(31536000)
        .includeSubDomains(true)
    )
);

```

9.4 Security Best Practices Checklist

Area	Best Practice
Passwords	Always hash with BCrypt (strength ≥ 10). Never store plain text or MD5/SHA1.
JWT Secret	Use 256-bit+ random secret. Store in env variable, never in source code.
JWT Expiry	Keep access tokens short-lived (15–60 min). Use refresh tokens for renewal.
HTTPS	Always use HTTPS in production. Enable HSTS header.
CORS	Whitelist specific origins. Never use allowedOrigins(*) with credentials.
CSRF	Enable for browser/cookie apps. Disable only for stateless JWT APIs.
Headers	Set Content-Security-Policy, X-Frame-Options, X-Content-Type-Options.
SQL Injection	Use parameterised queries / JPA. Never concatenate user input in SQL.
Logging	Log security events (failed logins, access denied). Never log passwords/tokens.
Dependencies	Regularly update Spring Security and dependencies. Check CVE databases.
Actuator	Restrict actuator endpoints in production. Require authentication.
Error Messages	Return generic error messages. Never expose stack traces to clients.
Rate Limiting	Implement login rate limiting to prevent brute-force attacks.
Refresh Tokens	Store refresh tokens in DB. Support revocation on logout.

Quick Reference – Spring Security Annotations & Classes

Annotation / Class	Purpose
@EnableWebSecurity	Enables Spring Security's web security support
@EnableMethodSecurity	Enables @PreAuthorize, @PostAuthorize, @Secured
@PreAuthorize(expr)	Evaluates SpEL before method executes
@PostAuthorize(expr)	Evaluates SpEL after method returns
@Secured({roles})	Restricts method to listed roles (no SpEL)
@WithMockUser	Test annotation; sets mock authentication in context
SecurityFilterChain	Defines the filter chain and HTTP security rules
UserDetailsService	Interface to load UserDetails by username
PasswordEncoder	Interface for encoding and matching passwords
BCryptPasswordEncoder	BCrypt implementation of PasswordEncoder
JwtAuthenticationConverter	Converts JWT claims to Spring GrantedAuthority objects
OncePerRequestFilter	Base class for custom security filters (e.g. JwtAuthFilter)
SecurityContextHolder	Static access to SecurityContext (current authentication)
OAuth2UserService	Loads OAuth2User from provider (implement for custom logic)
RegisteredClient	Represents an OAuth2 client in Spring Authorization Server
AuthorizationServerSettings	Configures Authorization Server endpoints and issuer URI

✓ Spring Security 6.x (Boot 3.x) uses the Lambda DSL for all configuration. The older and() chaining style is no longer needed. All configuration is done with .csrf(csrf -> ...) style lambdas.

■ Never disable CSRF for a cookie/session-based web app. Only disable it when your entire API is stateless (JWT) and no browser session cookies are used.

■ The complete OAuth2 ecosystem: Your Spring Boot App = Client + Resource Server. Your Spring Authorization Server = Authorization Server. Google/GitHub = External Authorization Servers. Your REST API endpoints = Protected Resources.